



Trailhead



[Apex Basics & Database \(/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)



Use sObjects

Use sObjects

Learning Objectives

After completing this unit, you'll be able to:

- Describe the relationship between sObjects and Salesforce records.
- Create and use specific sObject variables.
- Cast a generic sObject to a specific sObject.

Use sObjects

Because Apex is tightly integrated with the database, you can access Salesforce records and their fields directly from Apex. Every record in Salesforce is natively represented as an **sObject** in Apex. For example, the Acme account record corresponds to an Account sObject in Apex. The fields of the Acme record that you can view and modify in the user interface can be read and modified directly on the sObject as well.

The following table lists some populated fields of the Acme account example record. The Account sObject is an abstraction of the account record and holds the account field information in memory as an object.

Table 1. Account sObject for a Retrieved Record

Account Field	Value
Id	001D000000JIfXe
Name	Acme
Phone	(415)555-1212
NumberOfEmployees	100





- Account

Apex Basics & Database (/content/learn/modules/apex_database?

trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer).

- Opportunity

Use sObjects

If you've added custom objects in your organization, use the API Names of the custom objects in Apex. For example, a custom object called Merchandise corresponds to the Merchandise__c sObject in Apex.

Create sObject Variables

To create an sObject, you need to declare a variable and assign an sObject instance to it. The data type of the variable is the sObject type.

The following example creates an sObject of type Account with the name Acme and assigns it to the `acct` variable.

```
1 | Account acct = new Account (Name='Acme');
```



sObject and Field Names

Apex references standard or custom sObjects and their fields using their unique API Names.

API Names of objects and fields can differ from their labels. For example, the Employees field has a label of Employees and appears on the account record page as Employees but its API Name is NumberOfEmployees. To access this field in Apex, you'll need to use the API Name for the field: NumberOfEmployees.

The following are highlights of some rules used for API Names for custom objects and custom fields.

For custom objects and custom fields, the API Name always ends with the __c suffix. For custom relationship fields, the API Name ends with the __r suffix. For example:

- A custom object with a label of Merchandise has an API Name of Merchandise__c.
- A custom field with a label of Description has an API Name of Description__c.
- A custom relationship field with a label of Items has an API Name of Items__r.

In addition, spaces in labels are replaced with underscores in API Names. For example, a custom field name of Employee Seniority has an API Name of Employee_Seniority__c.

Find Object and Field Names



Setup, click the **Object Manager** tab to the right of the **Home** tab, and then click your object's name.

[Apex Basics & Database \(/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Create sObjects and Add Fields

Use sObjects

Before you can insert a Salesforce record, you must create it in memory first as an sObject. Like with any other object, sObjects are created with the `new` `copy`  operator:

```
1 | Account acct = new Account();
```



The API object name becomes the data type of the sObject variable in Apex. In this example, `Account` `copy`  is the data type of the `acct` `copy`  variable.

The account referenced by the `acct` `copy`  variable is empty because we haven't populated any of its fields yet. There are two ways to add fields: through the constructor or by using dot notation.

The fastest way to add fields is to specify them as name-value pairs inside the constructor. For example, this statement creates a new account sObject and populates its Name field with a string value.

```
1 | Account acct = new Account(Name='Acme');
```



The Name field is the only required field for accounts, which means that it has to be populated before being able to insert a new record. However, you can populate other fields as well for the new record. This example also adds a phone number and the number of employees.

```
1 | Account acct = new Account(Name='Acme', Phone='(415) 555-1212',  
    NumberOfEmployees=100);
```



Alternatively, you can use the dot notation to add fields to an sObject. The following is equivalent to the previous example, although it takes a few more lines of code.

```
1 | Account acct = new Account();  
2 | acct.Name = 'Acme';  
3 | acct.Phone = '(415) 555-1212';  
4 | acct.NumberOfEmployees = 100;
```

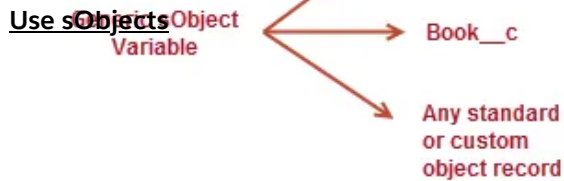


Work with the Generic sObject Data Type



record, whether it is a standard or custom object record.

[Apex Basics & Database \(/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)



This example shows how any Salesforce object, such as an account or a custom object called Book__c, can be assigned to a generic sObject variable.

```

1 | sObject subj1 = new Account(Name='Trailhead');
2 | sObject subj2 = new Book__c(Name='Workbook 1');
  
```



In contrast, variables that are declared with the specific sObject data type can reference only the Salesforce records of the same type.



Cast Generic sObjects to Specific sObject Types

When you're dealing with generic sObjects, you sometimes need to cast your sObject variable to a specific sObject type. One of the benefits of doing so is to be able to access fields using dot notation, which is not available on the generic sObject. Since sObject is a parent type for all specific sObject types, you can cast a generic sObject to a specific sObject. This example shows how to cast a generic sObject to Account.

```

1 | // Cast a generic sObject to an Account
2 | Account acct = (Account)myGenericSObject;
3 | // Now, you can use the dot notation to access fields on Account
4 | String name = acct.Name;
5 | String phone = acct.Phone;
  
```





In this unit, you've learned what sObjects are and how to use them. However, creating an sObject doesn't persist it as a record in the database. To save the sObject as a record, and do other things with it, use the Data Manipulation Language (DML). To retrieve a record, use the Salesforce Object Query Language (SOQL). Check out later units to learn about DML and SOQL.

Use sObjects Resources

- [Object Reference for the Salesforce Platform: Standard Objects](https://developer.salesforce.com/docs/atlas.en-us.object_reference.meta/object_reference/sforce_api_objects_list.htm) (https://developer.salesforce.com/docs/atlas.en-us.object_reference.meta/object_reference/sforce_api_objects_list.htm)
- [Apex Developer Guide: Working with sObjects](https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/langCon_apex_SObjects.htm) (https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/langCon_apex_SObjects.htm)
- [Apex Reference Guide: SObject Methods](https://developer.salesforce.com/docs/atlas.en-us.apexref.meta/apexref/apex_methods_system_object.htm#apex_System_SObject_methods) (https://developer.salesforce.com/docs/atlas.en-us.apexref.meta/apexref/apex_methods_system_object.htm#apex_System_SObject_methods)

Quiz

To complete this unit, you need to answer all the quiz questions correctly.

+100 Points

1 Describe the relationship between sObjects and Salesforce records.

- ☐ A The name of an sObject field in Apex is the label of the field in Salesforce.
- ☐ B A custom object's API Name and label are the same.
- ☐ C Every record in Salesforce is natively represented as an sObject in Apex.

2 How do you create an sObject instance, such as an Account?

- ☐ A Declare a variable and assign a new sObject instance to it.



Insert a persistent record into Salesforce using SOQL.

[Apex Basics & Database \(/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

3. Which of the following is correct about a generic sObject variable?

- A** A generic sObject variable can be assigned only to another generic sObject.
- B** Generic sObjects can't be created.
- C** A generic sObject variable can be assigned to any specific standard or custom sObject, such as Account or Book__c, using casting.

Second attempt earns 50 points. Three or more earns 25 points.

Check the Quiz to Earn 100 Points

Learn

Badges
Trails
Trailmixes
Superbadges
Trailhead Academy
Career Paths

Certifications

Certifications
Maintain Certifications
Verify Certifications

Community

Trailblazer Community
Events
Agentblazers
Salesforce Developers
Salesforce Admins
Code of Conduct
Report Illegal Content (EU)

Extras

Trailhead Login
Sales Enablement
Trailblazer Store
Salesforce Help

Download Trailhead GO





[Apex Basics & Database \(/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Use sObjects